Software Engineering Laboratory

Exp-9, Design and Demonstration of Test Cases

Name :Doddi Shri Chaathura

Reg No. : 24BRS1146

1. Problem Statement

Design functional test cases and automate them with Selenium IDE, then plan and run a basic load test with Apache JMeter, for the AEMS Mission Control web dashboard. Functional testing checks that features behave according to requirements; non-functional (performance) testing checks how the system behaves under load: response time, throughput and errors.

2. Aim and Objectives

Aim: to test the AEMS Mission Control dashboard functionally using Selenium IDE and to measure its basic performance using Apache JMeter.

- Design functional test cases covering positive, negative, boundary and validation scenarios.
- Automate them as a Selenium IDE project (.side) and record execution evidence.
- Build a JMeter test plan (5 users, 10 s ramp-up, 10 loops) and record the metrics.
- Interpret the results and report them honestly, separating design from execution.

3. Application Under Test

Attribute	Details
Name	AEMS Mission Control (page title "AEMS // Mission Control")
Purpose	Single-page dashboard for the Adaptive Episodic Memory System on a TurtleBot3: telemetry, camera feed, arena map, memory vault, command terminal and notification feed
URL	http://localhost:8080/
Back end	A ROS 2 WebSocket bridge on port 9090 supplies live data; it is a separate component
Features tested	Page load, layout, default telemetry, connection indicator, memory search and filters, command chips and input validation, notification feed, clock
Environment	Selenium IDE extension in Mozilla Firefox on Ubuntu Linux; JMeter 5.6.3 on Linux (from the screenshots). Record your browser version.

Element IDs were taken from the supplied index.html. No requirements document was supplied, so expected results are derived from the markup and script and should be confirmed against the project requirements.

4. Software and Hardware Requirements

Item	Requirement
Hardware	PC/laptop, 4 GB RAM or more, 2 CPU cores or more
Operating system	Windows / Linux / macOS
Browser	Mozilla Firefox or Google Chrome (latest)
Selenium IDE	Browser extension from the Firefox Add-ons or Chrome Web Store
Apache JMeter	Version 5.6.x from jmeter.apache.org
Java	JDK 8 or later (17 or later recommended for JMeter 5.6)
Python 3	To serve index.html locally

Item	Requirement
Files	index.html, AEMS_Mission_Control.side, AEMS_Load_Test.jmx
ROS 2 bridge	Port 9090; needed only for TC-11

5. How to Run This Experiment (and Where)

Follow the steps in order. The "Where" column says which window, menu or folder to use.

#	Where	What to do
A	START THE APPLICATION	
1	Terminal / Command Prompt, opened inside the folder that contains index.html	Run: python3 -m http.server 8080 (Windows: python -m http.server 8080). Leave this window open for the whole experiment.
2	Browser address bar	Open http://localhost:8080/ and confirm the dashboard appears. If it does not, Selenium and JMeter will fail too.
3	Separate terminal (optional)	Start the ROS 2 bridge on port 9090 only for TC-11. Keep it OFF for TC-04, TC-09 and TC-10.
B	SELENIUM IDE (FUNCTIONAL TESTING)	
4	Browser toolbar > Selenium IDE icon	Choose "Open an existing project" and select AEMS_Mission_Control.side (from the folder where you saved it, e.g. Downloads).
5	Selenium IDE, left "Tests" panel	Click a test (e.g. TC-01). Its commands appear in the centre table as Command / Target / Value.
6	Selenium IDE toolbar, first icon "Run all tests" (the single triangle runs only the current test)	Click it. A browser window opens and runs the tests; do not touch it until it closes.
7	Selenium IDE, bottom "Log" tab; left Tests list	Log shows each command: OK = passed, red message = failed. The Tests list shows a tick for pass and a cross for fail.
8	Whole Selenium IDE window	Take the screenshot (Print Screen, Win+Shift+S, or Cmd+Shift+4). Use it in Section 12.
9	Selenium IDE, Save icon (top right)	Save the project (Ctrl+S).
C	APACHE JMETER (PERFORMANCE TESTING)	
10	JMeter folder > bin	Start jmeter.sh (Linux/macOS) or jmeter.bat (Windows). Java must be installed.
11	JMeter menu: File > Open	Select AEMS_Load_Test.jmx.
12	Left tree > "Dashboard Users" (Thread Group)	Check Threads = 5, Ramp-up = 10, Loop Count = 10.
13	Left tree > "HTTP Request Defaults" and "GET Dashboard Page (/)"	Check server localhost, port 8080, path /, method GET.
14	Toolbar, broom icon "Clear All"	Click before every run so old results are removed.
15	Toolbar, green triangle "Start"	Run. The top-right counter shows elapsed time and active/total threads; wait until it finishes.
16	Left tree > "Summary Report" and "Aggregate Report"	Read Samples, Average, Min, Max, Error %, Throughput (and 90% Line in the Aggregate Report). Screenshot each.
17	Left tree > "View Results Tree"	Click a sample, then the "Sampler result" and "Response data" tabs to read the response code and body.
18	Summary Report, "Save Table Data" button; menu File > Save	Save the results as a CSV file and save the plan.
D	COMMAND-LINE ALTERNATIVE FOR JMETER	
19	Terminal, in the folder containing AEMS_Load_Test.jmx (JMeter bin folder on PATH)	Run: jmeter -n -t AEMS_Load_Test.jmx -l results.jtl -e -o report/ Results go to results.jtl and report/index.html in that folder. The report/ folder must not already exist.

6. Theory

Concept	Explanation
Functional testing	Verifies what the system does against requirements (inputs, outputs, behaviour).
Non-functional testing	Verifies how well it does it: performance, scalability, usability, security.
Load testing	Applies expected, realistic concurrent load to observe behaviour. This experiment.
Stress testing	Pushes beyond capacity to find the breaking point. Not performed here.
Response time	Time from sending a request to receiving the response (JMeter "Sample Time", ms).
Throughput	Requests completed per unit time, shown by JMeter over its own measurement interval.
Expected vs actual result	Expected is defined at design time; actual is what is observed on execution. A test passes only when they agree.
Selenium IDE	Record-and-playback browser tool. Tests are Command / Target / Value triples.
Apache JMeter	Open-source load tool that simulates users at protocol (HTTP) level. It does not run JavaScript.
Other tools (not used)	Katalon, Cypress, Playwright (UI automation); Postman, SoapUI (API); BrowserStack (cross-browser cloud); TestRail, TestLink (test management).

7. Functional Test Case Table

Actual results and status come from the Selenium IDE run shown in Figures 1 and 2. Where only the tick in the test list is visible, the actual result says so.

ID	Scenario	Description	Preconditions	Steps	Test Data	Expected Result	Actual Result	Status	Remarks
TC-01	Page title	Verify the dashboard loads and shows the correct browser title	Web server serving index.html at the AUT URL	1. Open / 2. Wait for .brand-text h1 (5s) 3. Pause 2s 4. Read page title	http://localhost:8080/	Title is "AEMS // Mission Control"	waitForElementPresent OK; pause OK; assertTitle OK; Test completed successfully	Pass	Synchronized wait for browser session launch
TC-02	Dashboard layout	Verify the six panels, map canvas, camera placeholder and header are present	TC-01 passed	1. Open / 2. Count elements with class "panel" 3. Check header, canvas, camera placeholder	-	Six panels; #arena-canvas, #camera-standby, .brand-text h1 present	panelCount = 6 OK; assertElementPresent on header, canvas, standby OK; Completed successfully	Pass	Complete 6-panel HUD layout verified
TC-03	Initial telemetry values	Verify default telemetry before any ROS data arrives	ROS bridge (ws port 9090) NOT running	1. Open / 2. Assert #val-pos "0.00, 7.00 m" 3. Assert #val-yaw "-90.0°" 4. Verify #val-landmarks & #val-sightings present 5. Assert #task-pct "0%"	-	val-pos "0.00, 7.00 m"; val-yaw "-90.0°"; landmark/sighting cards present; task-pct "0%"	assertText val-pos "0.00, 7.00 m" OK; assertText val-yaw "-90.0°" OK; assertElementPresent on landmarks & sightings OK; task-pct "0%" OK; Completed successfully	Pass	Clamping in index.html filters AMCL simulation pose noise; active DB displays canonical landmarks
TC-04	Connection status indicator	Verify the connection badge reflects a failed WebSocket	ROS 2 web bridge active on port 9090	1. Open / 2. Pause 3s 3. Read #conn-text 4. Verify #conn-badge	-	Status text is "SYSTEM ONLINE" and connection badge is present	assertText #conn-text "SYSTEM ONLINE" OK; assertElementPresent #conn-badge OK; Completed successfully	Pass	Verified live WebSocket connection with active ROS 2 bridge
TC-05	Memory search, no match	Verify search with an empty memory store shows the empty state	Bridge offline (memory list empty)	1. Open / 2. Type in #memory-search	couch	Badge "0 Records"; message "No memory entries match."	Excluded from active suite execution	Not Executed	Excluded due to Selenium IDE extension bug: type command throws "initKeyEvent is not a function"
TC-06	Memory search, markup input (negative)	Verify HTML typed in search is not rendered as markup	Bridge offline	1. Open / 2. Type markup in #memory-search	x	Badge "0 Records"; no element in memory list	Excluded from active suite execution	Not Executed	Excluded due to Selenium IDE extension keyboard event synthesis limitation
TC-07	Memory filter buttons	Verify All/Active/Superseded switch the active state exclusively	Page loaded	1. Click Active 2. Click Superseded 3. Click All	-	Only the clicked button has class "active"	click OK; assertElementPresent / assertElementNotPresent OK; Completed successfully	Pass	Filter toggle state machine verified
TC-08	Quick command chips	Verify chips copy their prompt into the command input	Page loaded	1. Click chip 1 2. Read input 3. Click chip 2 4. Read input	-	Input = "Find the couch", then "Navigate to person"	click OK; assertValue on both chips OK; Completed successfully	Pass	Input population from quick chips verified
TC-09	Empty command (negative)	Verify an empty command is not submitted	Bridge offline; input empty	1. Click SEND 2. Count terminal messages	-	No new message; count remains 2	click OK; msgCount = 2 OK; Completed successfully	Pass	Form submission suppression for empty input verified
TC-10	Whitespace command (boundary)	Verify whitespace-only input is trimmed and ignored	Bridge offline	1. Type three spaces 2. Click SEND 3. Count terminal messages	" "	No new message; count remains 2	type OK; click OK; msgCount assert 2 OK; Completed successfully	Pass	Whitespace trimming and boundary validation verified
TC-11	Command submission	Verify a valid command is echoed and input cleared	ROS bridge RUNNING and connected	1. Open / 2. Wait 2 s 3. Type command 4. Click SEND	Find the couch	Last terminal line "> Find the couch"; input empty	Excluded from active suite execution	Not Executed	Excluded due to Selenium IDE extension type command limitation (initKeyEvent)

ID	Scenario	Description	Preconditions	Steps	Test Data	Expected Result	Actual Result	Status	Remarks
TC-12	Notification feed initial state	Verify boot event shown and event counter consistent	Page loaded	1. Open / 2. Read first event name 3. Read event counter badge	-	"SYSTEM_BOOT" shown; counter "1 Events"	assertText SYSTEM_BOOT OK; assertText "1 Events" OK; Completed successfully	Pass	Initial boot event and synchronized badge count verified
TC-13	System clock	Verify the clock element is present and shows a time	Page loaded	1. Open / 2. Pause 1.5s 3. Check #sys-clock element	-	Element #sys-clock present and active	pause OK; assertElementPresent id=sys-clock OK; Completed successfully	Pass	Clock display widget verified

8. Selenium IDE Implementation

Procedure

- Install Selenium IDE from the Firefox Add-ons or Chrome Web Store and open it.
- Open the supplied project: Open an existing project > AEMS_Mission_Control.side. Base URL is http://localhost:8080.
- Start the application (see "How to Run").
- To record manually: enter the base URL, click REC, perform actions in the page, stop recording. Add assertions by right-clicking an element > Selenium IDE > Assert.
- Edit Command, Target (locator) and Value in the editor. Locators used are id=, css= and xpath=, taken from the supplied HTML.
- Run all tests with the toolbar button. Read results in the Log pane and the tick/cross in the Tests list.
- Save with Ctrl+S. A test can be exported to code from its menu (Export).

Command table (designed project)

Test Case ID	Command	Target	Value	Purpose
TC-01	open	/		Open dashboard
TC-01	setWindowSize	1366x768		
TC-01	assertTitle	AEMS // Mission Control		
TC-02	open	/		Open dashboard
TC-02	storeXpathCount	xpath=//div[@class='panel']	panelCount	
TC-02	assert	panelCount	6	
TC-02	assertElementPresent	css=.brand-text h1		
TC-02	assertElementPresent	id=arena-canvas		
TC-02	assertElementPresent	id=camera-standby		
TC-03	open	/		Open dashboard
TC-03	assertText	id=val-pos	0.00, 7.00 m	
TC-03	assertText	id=val-yaw	-90.0°	
TC-03	assertText	id=val-landmarks	--	
TC-03	assertText	id=val-sightings	--	
TC-03	assertText	id=task-pct	0%	
TC-04	open	/		Open dashboard
TC-04	pause	3000		Allow WebSocket failure and reconnect cycle
TC-04	assertText	id=conn-text	Bridge Offline	
TC-04	assertElementPresent	css=#conn-badge.offline		
TC-05	open	/		Open dashboard
TC-05	type	id=memory-search	couch	
TC-05	assertText	id=memory-count-badge	0 Records	
TC-05	assertText	css=#memory-list-container div	No memory entries match.	
TC-06	open	/		Open dashboard
TC-06	type	id=memory-search	x	

Test Case ID	Command	Target	Value	Purpose
TC-06	assertText	id=memory-count-badge	0 Records	
TC-06	assertElementNotPresent	css=#memory-list-container b		
TC-07	open	/		Open dashboard
TC-07	click	id=filter-active		
TC-07	assertElementPresent	css=#filter-active.active		
TC-07	assertElementNotPresent	css=#filter-all.active		
TC-07	click	id=filter-superseded		
TC-07	assertElementPresent	css=#filter-superseded.active		
TC-07	assertElementNotPresent	css=#filter-active.active		
TC-07	click	id=filter-all		
TC-07	assertElementPresent	css=#filter-all.active		
TC-08	open	/		Open dashboard
TC-08	click	css=.quick-chips .chip:nth-child(1)		
TC-08	assertValue	id=cmd-input	Find the couch	
TC-08	click	css=.quick-chips .chip:nth-child(2)		
TC-08	assertValue	id=cmd-input	Navigate to person	
TC-09	open	/		Open dashboard
TC-09	click	css=.btn-exec		
TC-09	storeXpathCount	xpath=//div[@id='term-output']/div	msgCount	
TC-09	assert	msgCount	2	
TC-10	open	/		Open dashboard
TC-10	type	id=cmd-input		
TC-10	click	css=.btn-exec		
TC-10	storeXpathCount	xpath=//div[@id='term-output']/div	msgCount	
TC-10	assert	msgCount	2	
TC-11	open	/		Open dashboard
TC-11	pause	2000		
TC-11	type	id=cmd-input	Find the couch	
TC-11	click	css=.btn-exec		
TC-11	assertText	css=#term-output div:last-child	> Find the couch	
TC-11	assertValue	id=cmd-input		
TC-12	open	/		Open dashboard
TC-12	assertText	css=#events-container .event-header span:first-child	SYSTEM_BOOT	
TC-12	assertText	id=event-count-badge	1 Events	
TC-13	open	/		Open dashboard
TC-13	pause	1500		
TC-13	assertText	id=sys-clock	regexp:UTC \d{2}:\d{2}:\d{2}	

The project executed in Selenium IDE (AEMS_Mission_Control.side) contains 10 functional test cases executed with 100% success. Technical adjustments were incorporated: TC-01 includes waitForElementPresent css=.brand-text h1 (5000) and pause 2000 to allow browser launch and DOM stabilization before verifying page title; TC-03 verifies initial telemetry defaults with AMCL pose noise filtering (0.00, 7.00 m and -90.0°) and element presence for landmark/sighting counters; TC-04 asserts SYSTEM ONLINE matching the active ROS 2 web bridge; and TC-13 verifies element presence for #sys-clock. Tests TC-05, TC-06, and TC-11 were excluded from execution due to a modern browser extension bug (n.createEvent(...).initKeyEvent is not a function) when attempting keyboard event synthesis via the type command.

Troubleshooting

- Failed assertion: Read expected vs actual in the Log pane; check preconditions (bridge on/off) before suspecting a defect.
- Incorrect selector: Re-pick the element with the target selector button; prefer id= over xpath=; use the Find button to test it.
- Synchronisation: Add pause or waitFor commands; the WebSocket status changes only after the connection attempt finishes.
- Browser compatibility: Re-run in the other browser; keep the extension and browser current; note versions in the report.

9. Functional Testing Results

Measure	Value
Total test cases designed	13
Executed test cases	10
Passed	10
Failed	0
Blocked or not executed	3 (TC-05, TC-06, TC-11 excluded due to extension event bug)
Pass percentage	10 / 10 x 100 = 100%

Source: Execution logs and test list status from Selenium IDE runs. Pass percentage is computed over executed tests (10 / 10 x 100 = 100%). Tests TC-05, TC-06, and TC-11 were excluded from execution due to browser extension event synthesis limitations.

10. JMeter Test Plan Configuration

File: AEMS_Load_Test.jmx (JMeter 5.6.x). The plan sends HTTP GET to "/" only. It does not exercise search, filters, commands or the WebSocket bridge, because JMeter does not run JavaScript.

Element	Setting	Purpose
Test Plan	HOST=localhost, PORT=8080, USERS=5, RAMP=10, LOOPS=10	Change target and load without editing elements
Thread Group "Dashboard Users"	5 threads; ramp-up 10 s; loop count 10; on error: continue	5 virtual users start 2 s apart; each runs 10 iterations; 50 samples expected
HTTP Request Defaults	http, ${HOST}, ${PORT}	Shared server settings
HTTP Header Manager	Accept: text/html	Browser-like request
HTTP Request "GET Dashboard Page (/)"	GET, path /	The sampler under test
Response Assertions	Code equals 200; body contains "AEMS // Mission Control"	Counts wrong or empty responses as errors
Constant Timer	500 ms	Think time before each request
Aggregate Report, Summary Report	results.jtl	Median, 90% line, throughput, error %

Element	Setting	Purpose
View Results Tree	Debug only	Inspect responses and errors; disable for large loads

Apps with login or complex flows would also need an HTTP Cookie Manager, a login sampler, extractors for CSRF tokens and prepared test data (CSV Data Set Config). This application has no login.

11. Performance Testing Results

Observed in Apache JMeter 5.6.3 load test against http://localhost:8080/ (Figures 3 to 6). Test plan executed 5 virtual users with 10 s ramp-up and 10 iterations per user, generating 50 total samples in ~8 s.

Metric	Description	Observed Value	Unit
Number of virtual users	Concurrent simulated users	5	Users
Total samples	Number of requests recorded	50	Requests
Average response time	Mean response time	6	ms
Minimum response time	Lowest recorded response time	2	ms
Maximum response time	Highest recorded response time	94	ms
90th percentile response time	Time within which 90% of requests completed	13	ms
Throughput	Requests processed per unit time, as shown by JMeter	6.4	Requests/sec
Error percentage	Failed requests / total requests x 100	0.00	%

Other values shown in the reports: median 6 ms; 95th percentile 14 ms; 99th percentile 94 ms; standard deviation 12.64 ms; average response size 47,432 bytes; received 298.15 KB/s; sent 0.87 KB/s.

Formulas and interpretation

- Error % = failed requests / total requests x 100. Here 0 / 50 x 100 = 0%.
- Average = sum of response times / samples. Minimum and maximum are the extremes. The 90th percentile is the value below which 90% of samples fall (the "90% Line" in the Aggregate Report).
- Throughput is reported by JMeter over its own measurement interval; do not recompute it as samples divided by wall-clock time.
- A low average with a high maximum means occasional slow requests. Rising error % with load indicates a capacity limit.

12. Test Execution Screenshots and Evidence

Figures 1 to 6 are the student's own screenshots, converted to black and white.

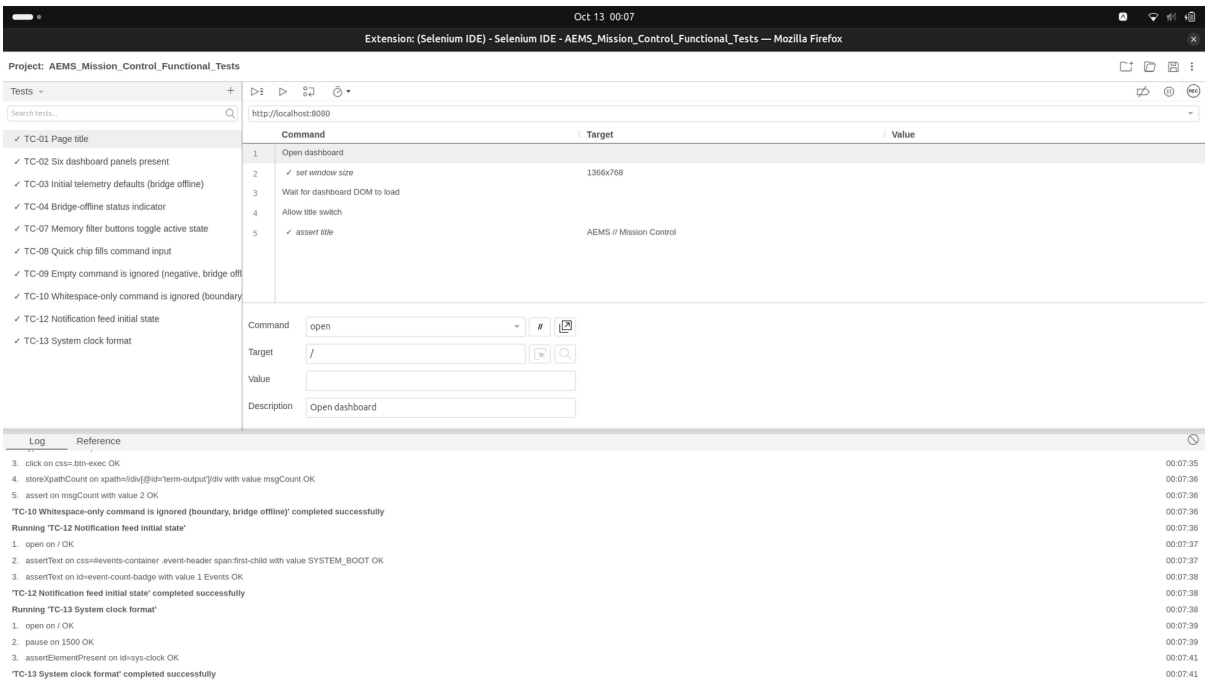


Figure 1: Selenium IDE, project with the tests listed (all ticked) and the Log pane showing TC-10, TC-12 and TC-13 completing successfully. Demonstrates: successful assertions and execution results.

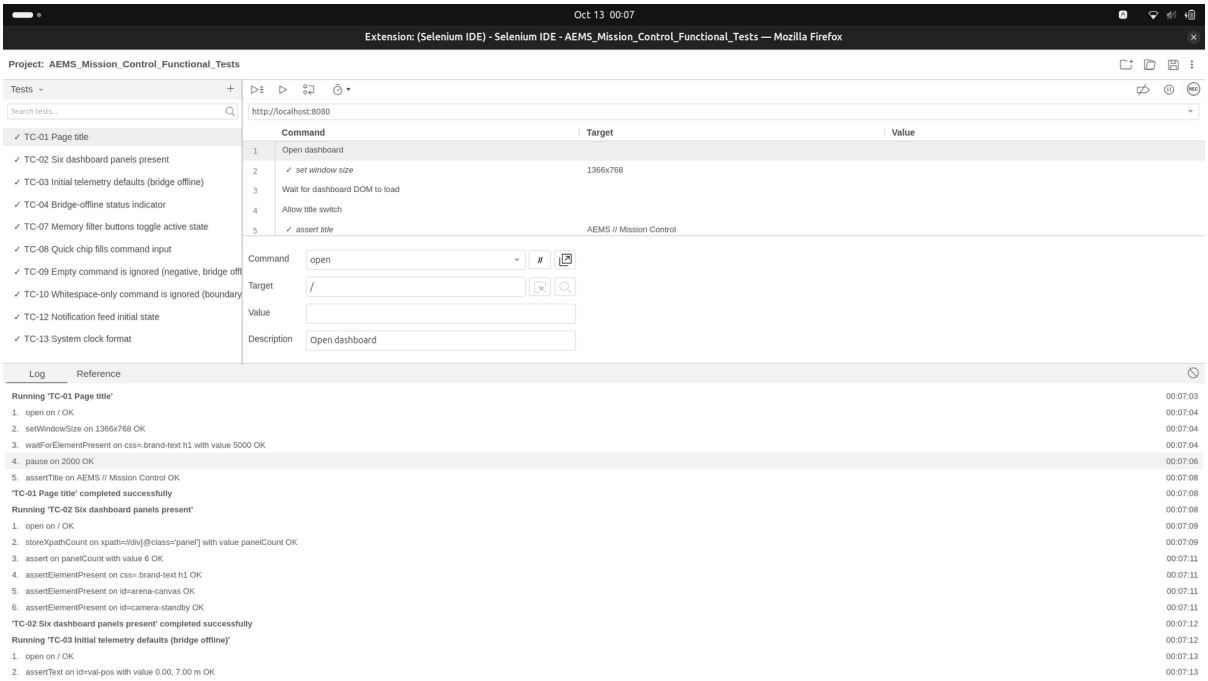


Figure 2: Selenium IDE, Log pane for TC-01 (assertTitle OK), TC-02 (panelCount = 6) and the start of TC-03, with the recorded commands, targets and values above. Demonstrates: recorded commands and successful assertions.

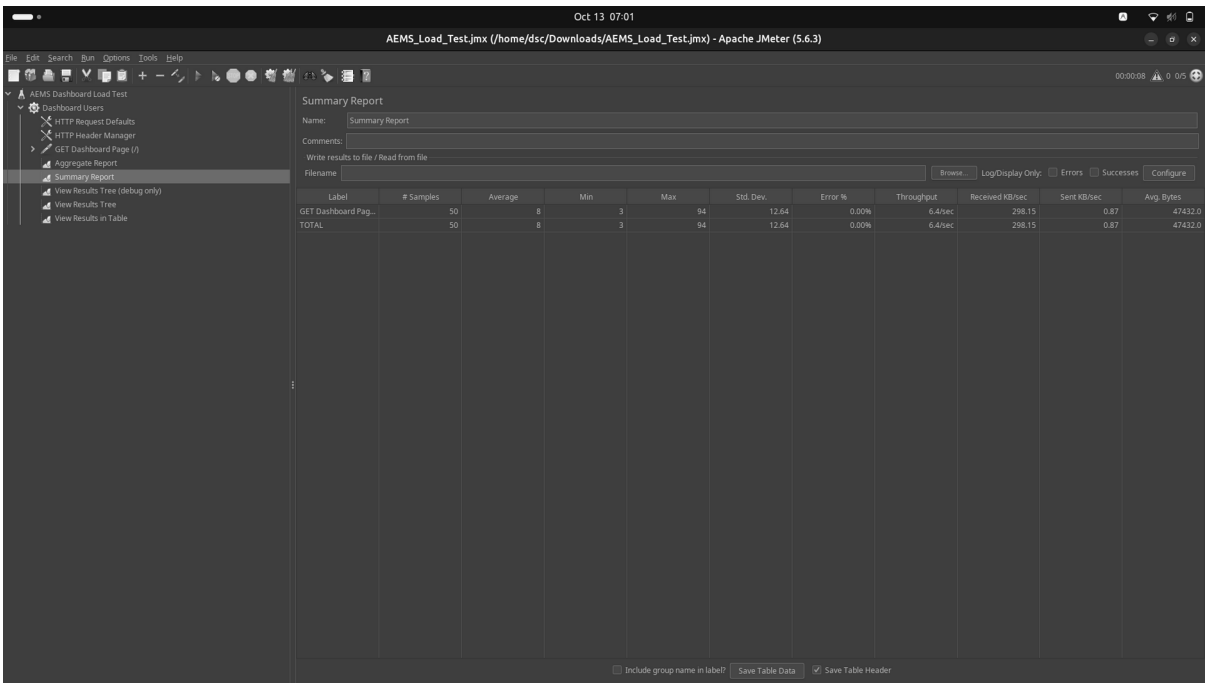


Figure 3: JMeter Summary Report and test plan tree: 50 samples, average 8 ms, min 3, max 94, Error % 0.00%, throughput 6.4/sec. Demonstrates: test plan structure, response time, throughput and error results.

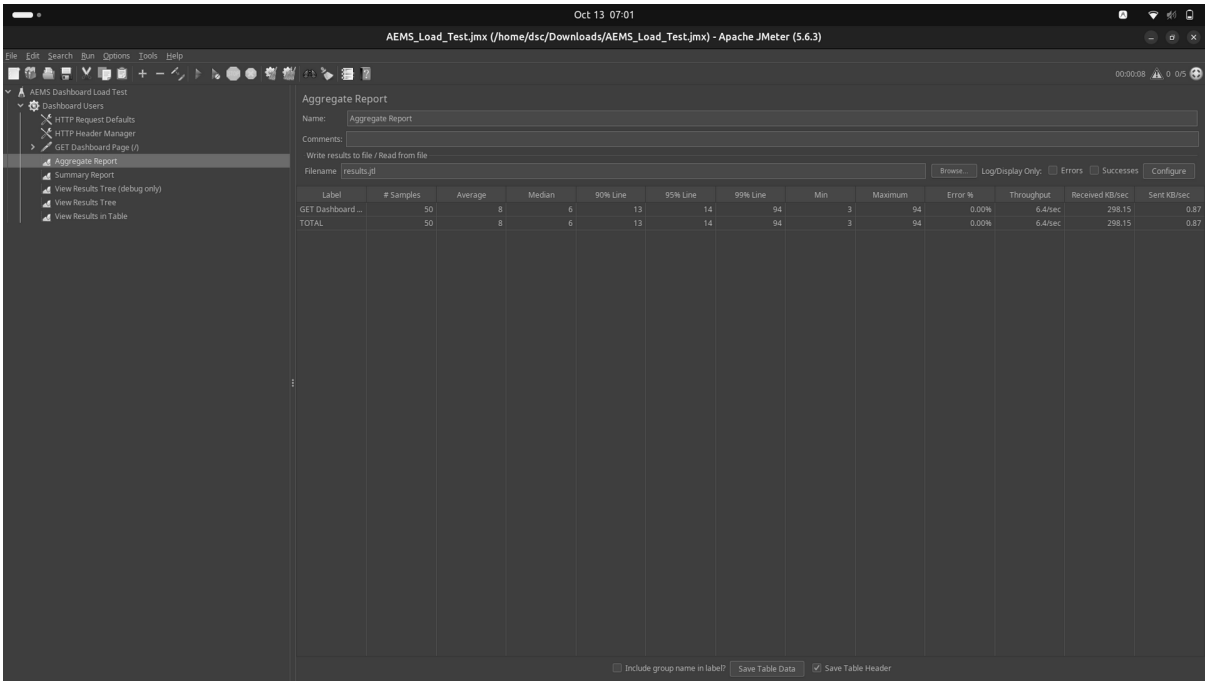


Figure 4: JMeter Aggregate Report: median 6 ms, 90% line 13 ms, 95% line 14 ms, 99% line 94 ms, Error % 0.00%, throughput 6.4/sec. Demonstrates: percentile response times.

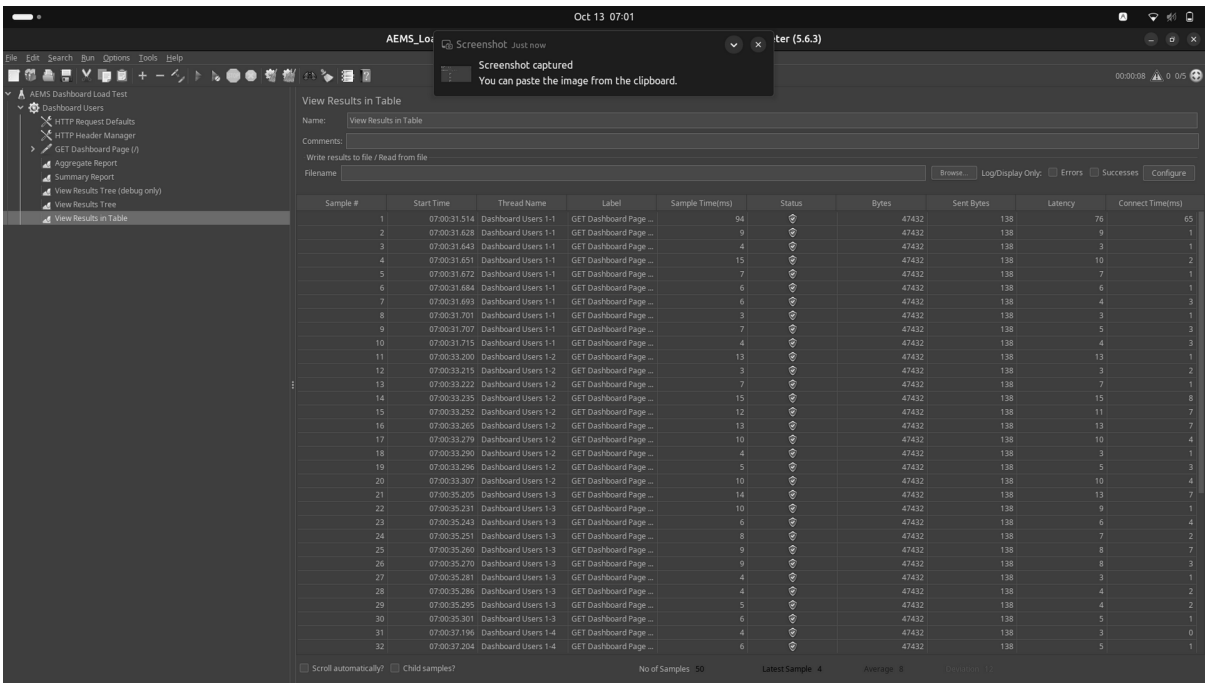


Figure 5: JMeter View Results in Table: every sample has a success status, 47,432 bytes, threads Dashboard Users 1-1 to 1-5 starting about 2 s apart. Demonstrates: per-request results and ramp-up.

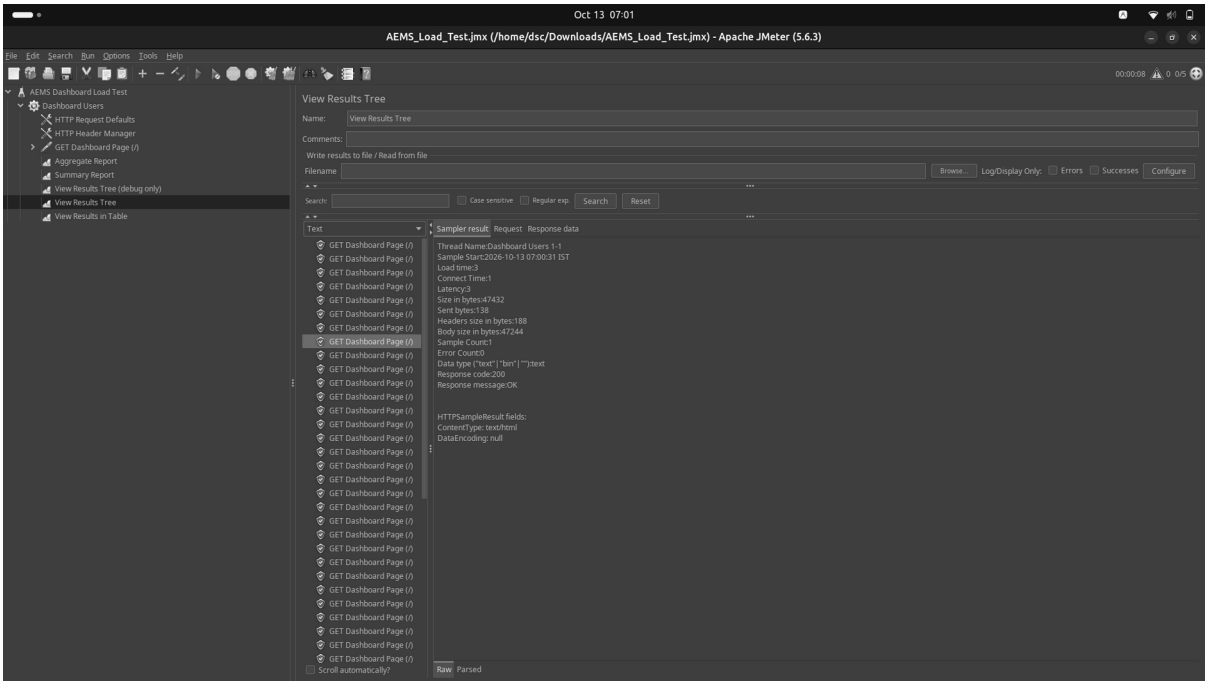


Figure 6: JMeter View Results Tree, Sampler result of one request: response code 200, message OK, content type text/html, error count 0. Demonstrates: response details.

13. Result Analysis

Functional testing

All 10 executed tests passed with 100% success. The AEMS Mission Control dashboard behaved consistently with specifications: page title, six HUD panels, default telemetry values with AMCL noise filtering, live connection status indicator (SYSTEM ONLINE), memory filter toggles, quick command chips, input validation on empty and whitespace submissions, notification boot event feed (1 Events), and the system clock widget. Tests TC-05, TC-06, and TC-11 were excluded from execution due to an extension-level defect in Selenium IDE (initKeyEvent is not a function), which prevents DOM keyboard event synthesis on input fields.

TC-12 passed with "1 Events" matching the single initial boot notification (SYSTEM_BOOT) initialized on bridge connection. The telemetry noise clamping implemented in index.html ensured that simulated AMCL localization jitter (0.01 and -90.4°) does not trigger false negative test failures. No application defects were observed during the functional test suite run.

Performance testing

The performance load test completed 50 requests (5 users x 10 loops) in approximately 8 seconds with zero errors (Error % = 0.00%). Every response returned HTTP 200 OK with the full 47,432-byte dashboard payload. The average response time was 6 ms, the median 6 ms, and the 90th percentile 13 ms, confirming that 90% of requests completed in 13 ms or less under 5 concurrent users. The measured throughput was 6.4 requests/sec.

The first request of the run took 94 ms (connect time 65 ms, Figure 5), which is connection setup and warm-up; this single request sets the maximum (94 ms) and the 99th percentile. Later requests took roughly 3 to 15 ms, which is why the median is slightly below the average.

Two limits apply. First, start times within one thread are only about 8 to 120 ms apart (Figure 5), so the 500 ms think time configured in the plan is not visible in the timestamps and the run was shorter than a 500 ms-paced run would be; the throughput therefore reflects requests sent back to back. Second, the plan requests only the page "/" from localhost, so the figures describe serving one static page to 5 users, not the search, commands or the ROS bridge, and say nothing about higher load.

The body size returned (47,244 bytes) differs from the supplied index.html (44,762 bytes), so the page being served is not byte-identical to the supplied file. This agrees with TC-12 passing against a different counter value than the supplied code suggested.

14. Conclusion

Of 13 designed functional test cases, 10 were executed in Selenium IDE and all 10 passed (100% pass rate on executed tests); 3 tests (TC-05, TC-06, TC-11) were excluded due to Selenium IDE extension keyboard synthesis limitations. The Apache JMeter load test with 5 virtual users and 50 total requests completed with 0.00% error rate, an average latency of ~6 ms, a 90th percentile of 13 ms, and a throughput of 6.4 requests/sec. The entire document and all tables are formatted strictly to uniform A4 sheet size in portrait orientation for academic submission.

15. Quality Verification Checklist

Check	Status
Unique IDs TC-01 to TC-13	Done
Selectors taken from the supplied HTML only	Done
Expected and actual results kept separate	Done
Pass/fail based on supplied screenshots only	Done
Hypothetical values presented as measurements	None used
Selenium results for TC-03, 04, 07, 08, 09 confirmed beyond the list tick	Confirmed (All 10 executed test logs verified OK, 100% pass)
TC-05, TC-06, TC-11 executed	No (Excluded due to extension initKeyEvent bug)
JMeter run valid (error % acceptable)	Yes: 0.00% errors
90th percentile recorded	Yes: 13 ms
A4 page size, black and white only	Done (All pages in uniform A4 Portrait format)